priority queue with all the undiscovered points in the neighborhoods of any point in C, then we can be sure that:

1. The reachability distance of all the points at the frontier of the cluster (all the points that are neighbors to at least one point in the cluster) is correctly stored;
2. The value stored is the smallest of the reachability distances from any of the points already processed to q (although we only care about current cluster's);
3. The top of the queue holds the closest point to cluster C;

    a) Technically, at the top of the queue we have the point q with associated a value εq (the smallest value for ε for which q would still be reachable from C) such that, considering any other points w in C's frontier, $\varepsilon q \leq \varepsilon w$;

4. Therefore, any of the points still to be processed will have at least the same reachability distance from C, or higher.

Listing 12.7 and 12.8 describe the main OPTICS algorithm, while on the book's repo, you can find a Python implementation of the algorithm and this Jupyter notebook to experiment with it.

Listing 12.7 OPTICS Clustering

```
function optics(points, epsMax, minPoints)
                    #1
  reachabilityDistances ← null (" p ∈
points)                                    #2
```